Apache Kafka – Detailed Architecture & Working

1. What is Kafka?

Apache Kafka is a **distributed event streaming platform** that allows applications to:

- Publish events
- Store events
- Subscribe to events

Kafka is commonly used in **Event Driven Architecture (EDA)**.

In EDA we already saw:

- Pub/Sub Model
- Streaming Model
- Advantages of EDA

Today we focus on the **most popular streaming platform: Kafka**.

---

2. Kafka Core Components

2.1 Producer

A **Producer** is an application that **publishes events to Kafka**.

Key points:

- Producer sends events to Kafka topics.

- Producer **does not care who will consume the event**.
- Multiple producers can publish to the same topic.

Example:

Producer → Kafka → Consumers

2.2 Topic

A **Topic** is a **logical category or folder** where events are stored.

Example:

Topic: order-events

Multiple producers can write events to the same topic.

Example:

Producer1 → order-events
Producer2 → order-events

---

3. Partitions

A **Partition** is:

- A **physical**
- **Ordered**
- **Append-only log**

Each topic can have **multiple partitions**.

Example:

Topic: order-events

Partition 0

Partition 1
Partition 2

---

4. Partitioning Strategy

Producer decides **which partition an event should go to**.

4.1 Key Based Partitioning

partition = hash(key) % number_of_partitions

Example:

Event	Key	Partition
E0	123	1
E1	456	2
E2	789	0

Advantage:

- **Ordering guaranteed for same key**

All events with the same key go to the **same partition**.

4.2 Round Robin Partitioning (No Key)

If no key is provided, Kafka distributes events evenly.

Example:

Event	Partition
E0	P0
E1	P1

E2 P2

E3 P0

Advantage:

- Even distribution of load.

Disadvantage:

- Ordering not guaranteed.

4.3 Custom Partitioning

Developers can implement custom logic.

Example:

country == "India" → Partition 0
country == "US" → Partition 1

---

5. Event Ordering

Inside a partition:

- Events are **written sequentially**
- Each event receives an **offset**

Example:

Partition 0

Offset 1 → Event0
Offset 2 → Event3

Kafka guarantees:

✅ **Ordering inside a partition**

Kafka does NOT guarantee:

❌ **Global ordering across partitions**

Example:

Event4 in Partition1
may happen before or after
Event0 in Partition0

---

6. Kafka Log Storage

Events are stored in **log files on disk**.

Example:

Topic: order-events

Partition0 → order-events-0.log
Partition1 → order-events-1.log
Partition2 → order-events-2.log

Each new event:

1. Gets next offset
2. Appended to log file

---

7. Log Segments

Kafka does not store the entire partition in one file.

It divides logs into **segments**.

Example:

Partition0

000000.log → offsets 0–499
000500.log → offsets 500–999
001000.log → offsets 1000+

Benefit:

- Faster lookup
- Easier deletion
- Better storage management

---

8. Segment Index

Each segment has an **index file**.

Example:

000000.index

Structure:

Offset	File Position
0	0
150	4200
300	8900
450	13200

Not every offset is indexed.

Kafka indexes **every N bytes**.

Benefit:

- Faster lookup

- Reduced memory usage

---

9. Kafka Broker

A **Broker** is a **single Kafka server instance**.

Responsibilities:

- Stores partition data
- Serves producers and consumers

Important:

- Brokers store **some partitions of some topics**
- Topics and partitions are **distributed across brokers**

---

10. Consumers

A **Consumer** reads events from Kafka.

Consumers usually work in **Consumer Groups**.

---

11. Consumer Groups

A **Consumer Group** is a set of consumers working together.

Rule:

> Inside one consumer group, a partition is read by only one consumer.

Scenario 1: Consumers = Partitions

Topic: order-events (3 partitions)

Consumers: 3

Assignment:

Consumer1 → Partition0
Consumer2 → Partition1
Consumer3 → Partition2

Scenario 2: Consumers < Partitions

Partitions: 6
Consumers: 3

Assignment:

Consumer1 → P0,P3
Consumer2 → P1,P4
Consumer3 → P2,P5

Scenario 3: Consumers > Partitions

Partitions: 3
Consumers: 5

Assignment:

Consumer1 → P0
Consumer2 → P1
Consumer3 → P2
Consumer4 → IDLE
Consumer5 → IDLE

---

12. Multiple Consumer Groups

Different consumer groups can read the same partition.

Example:

Partition0

notification-service
analytics-service
audit-service

Each group maintains its own offset.

---

13. Consumer Offsets

Offsets are stored in an internal topic:

_consumer_offsets

Example record:

Consumer Group	Topic	Partition	Offset
notification	order-events	0	105
analytics	order-events	0	60

Partition selection:

hash(groupId) % 50

Default partitions:

_consumer_offsets = 50 partitions

---

14. Offset Commit Strategies

Auto Commit

auto.commit.interval.ms = 5000

Commit every 5 seconds.

Problem:

Possible **data loss**.

Example:

Consumer processed 50 events
Auto commit happened at 99
Crash occurs

Events 51–99 lost

Delivery type:

At most once

Manual Commit

Steps:

1. Poll events
2. Process all events
3. Commit offset

If crash occurs:

Before commit → **events reprocessed**

Delivery type:

At least once

---

15. Kafka Cluster

A **Kafka Cluster** is a group of brokers.

Benefits:

Scalability

Distribute load across servers.

Fault Tolerance

Cluster works even if some brokers fail.

High Availability

No single point of failure.

---

16. Partition Replication

Each partition has:

- **1 Leader**
- **Multiple Followers**

Example:

Topic: order-events
Partitions: 3
Replication Factor: 2

Example distribution:

Broker1
P0 Leader
P1 Follower

Broker2
P1 Leader
P2 Follower

Broker3
P0 Follower
P2 Leader

---

17. Leader Responsibilities

Leader handles:

- Producer writes
- Consumer reads
- Partition log
- Follower coordination

---

18. Follower Responsibilities

Followers:

- Replicate leader data
- Stay in sync
- Can become leader on failure

Followers **do not serve clients**.

---

19. Controller

A **Controller** is a special broker responsible for cluster coordination.

Responsibilities:

- Topic creation

- Partition leader election
- Detect broker failures
- Manage cluster metadata

---

20. KRaft (Kafka Raft)

Older Kafka used **ZooKeeper**.

Modern Kafka uses:

KRaft (Kafka Raft)

Benefits:

- Built into Kafka
- No external dependency
- Simpler deployment

---

21. Consensus in KRaft

Kafka uses **Raft consensus algorithm**.

Decisions require **Quorum (Majority)**.

Example:

Controllers = 3
Quorum = (3/2) + 1 = 2

At least **2 controllers must agree**.

Used for:

- Leader election
- Metadata updates

---

22. Producer Write Flow

Steps:

1. Producer creates event
2. Requests metadata
3. Calculates partition
4. Finds leader broker
5. Sends event to leader
6. Leader writes event to log
7. Followers replicate
8. Leader sends ACK

Example:

acks = all

Leader waits for **all ISR replicas**.

---

23. Consumer Read Flow

Steps:

1. Consumer joins group

2. Group coordinator assigns partitions
3. Consumer fetches last committed offset
4. Consumer reads new events
5. Consumer processes events
6. Consumer commits offset
7. Loop continues

Kafka uses:

PULL model

Consumers request data.

Kafka Consumer Flow

The consumer reads messages from a Kafka topic in the following sequence.

● Consumer Starts

When the consumer application starts, it connects to any broker in the Kafka cluster.

Example config:

bootstrap.servers=broker1:9092

group.id=order-consumer-group

The **consumer group ID** identifies which group the consumer belongs to.

● Find the Group Coordinator

The broker calculates which partition of the internal topic **_consumer_offsets** manages this consumer group.

Formula:

hash(group.id) % number_of_partitions

Then Kafka finds the **leader of that partition**, and that broker becomes the **Group Coordinator**.

● Consumer Joins the Group

The consumer sends a **JoinGroup request** to the **Group Coordinator**.

The coordinator:

- Registers the consumer
- Tracks active consumers
- Prepares for partition assignment

● Partition Assignment (Rebalance)

If multiple consumers exist in the group, Kafka performs **rebalance**.

The coordinator distributes partitions among consumers.

Example topic:

Topic: orders

Partitions: 4

Consumers: 2

Assignment might look like:

Consumer	Partitions
Consumer 1	0,1
Consumer 2	2,3

This ensures **each partition is read by only one consumer in the group**.

● Consumer Finds Partition Leader

For each assigned partition, the consumer asks Kafka metadata to find the **leader broker**.

Example:

Partition	Leader
orders-0	Broker1
orders-1	Broker2

The consumer then connects **directly to those leaders**.

● Poll Messages

The consumer continuously calls:

consumer.poll()

Kafka returns records starting from the **last committed offset**.

Example message:

Partition	Offset	Message
orders-0	120	OrderCreated

● Process the Message

Your application processes the event.

Example:

Process payment

Update database

Send email

● Commit Offset

After successful processing, the consumer commits the offset.

Example:

consumer.commitSync();

Kafka writes this offset as a record in the internal topic:

_consumer_offsets

This ensures that if the consumer restarts, it continues from the correct offset.

- **Next Poll**

Consumer polls again:

consumer.poll()

Kafka returns the **next batch of records**.

This loop continues indefinitely.

Complete Flow Diagram

Consumer Starts

↓

Connect to Kafka Broker

↓

Find Group Coordinator

↓

Join Consumer Group

↓

Partition Assignment (Rebalance)

↓

Find Partition Leaders

↓

Poll Messages

↓

Process Messages

↓

Commit Offset to _consumer_offsets

↓

Repeat

Example Real Scenario

Topic **orders**

Partitions = 3

Consumer Group = order-service

Consumers = 2

Assignment:

Consumer1 → orders-0, orders-1

Consumer2 → orders-2

Flow:

Consumer1 polls orders-0 & orders-1

Consumer2 polls orders-2

Both commit offsets to **_consumer_offsets**.

Important Guarantee

Kafka guarantees:

- **Each partition is consumed by only one consumer in a group**

- **Offsets are persisted**
- **Consumers can resume after restart**

---

24. ISR (In Sync Replica)

ISR = replicas fully caught up with leader.

Example:

Leader: Broker1
Followers: Broker2, Broker3

ISR = {1,2,3}

If Broker3 lags:

ISR = {1,2}

---

25. Producer Acknowledgment Modes

ack = 0

Fire and forget.

No confirmation.

ack = 1

Leader confirms.

ack = all

All ISR replicas confirm.

Safest option.

---

26. Log Cleanup Policies

Delete Policy

Default cleanup strategy.

cleanup.policy = delete

Data deleted based on:

- Time
- Size

Example:

retention.ms = 7 days
retention.bytes = 1GB

Compaction Policy

cleanup.policy = compact

Keeps **latest value per key**.

Example:

Before:

user1 v1
user1 v2

After compaction:

user1 v2

Log Compaction in Kafka

Log compaction is a Kafka feature that keeps **only the latest value for each key in a topic**, instead of keeping every message forever.

This helps maintain a **cleaned log with the most recent state of each key**.

Example Without Log Compaction

Suppose a topic stores user profile updates.

Offset	Key (UserId)	Value
0	user1	name=John
1	user2	name=Sam
2	user1	name=John Smith
3	user2	name=Samuel

If **log compaction is disabled**, Kafka keeps **all records**.

Example With Log Compaction

With log compaction enabled, Kafka eventually removes older records with the same key.

Final compacted log:

Offset	Key	Value
2	user1	name=John Smith
3	user2	name=Samuel

Only the **latest record per key** remains.

Where Log Compaction Is Used

Log compaction is often used in topics where the **latest state matters more than the full history**, such as:

- User profile updates
- Configuration topics
- Cache rebuild topics

Kafka also uses it internally for the topic **_consumer_offsets** to keep only the **latest committed offset per consumer group and partition**.

How Log Compaction Works

Kafka runs a background process called the **log cleaner**.

Steps:

1. Kafka scans the log segments of a compacted topic.
2. It keeps the **latest record for each key**.
3. Older records with the same key are removed.

Important rule:

> Compaction works **only when messages have keys**.

If a record has **no key**, it cannot be compacted.

Topic Configuration

To enable log compaction:

cleanup.policy=compact

Example topic configuration:

cleanup.policy=compact
min.cleanable.dirty.ratio=0.5

This means Kafka will compact when **50% of the log is dirty (contains duplicate keys)**.

Log Compaction vs Log Retention

Feature	Log Retention	Log Compaction
Purpose	Delete old messages by time/size	Keep latest record per key
Deletes data	Based on time or size	Based on duplicate keys
Use case	Event streams	State storage

Example Real Use Case

Imagine a topic storing **product prices**.

Messages:

Key	Value
product1	100
product1	110
product1	120

After compaction:

Key	Value
product1	120

Consumers starting later will still see **the latest price**.

How Log Compaction Helps

1. Saves Storage

Old duplicate records are removed.

2. Faster State Recovery

A new consumer can rebuild state quickly because only the latest records remain.

3. Enables Stateful Systems

Systems like caches or databases can rebuild the **latest state from Kafka**.

---

27. Kafka Performance Optimizations

Page Cache

Kafka relies on **OS page cache**.

Write flow:

Kafka → Page Cache (RAM) → Disk

Benefits:

- Fast writes
- No blocking disk IO

Zero Copy

Normal flow:

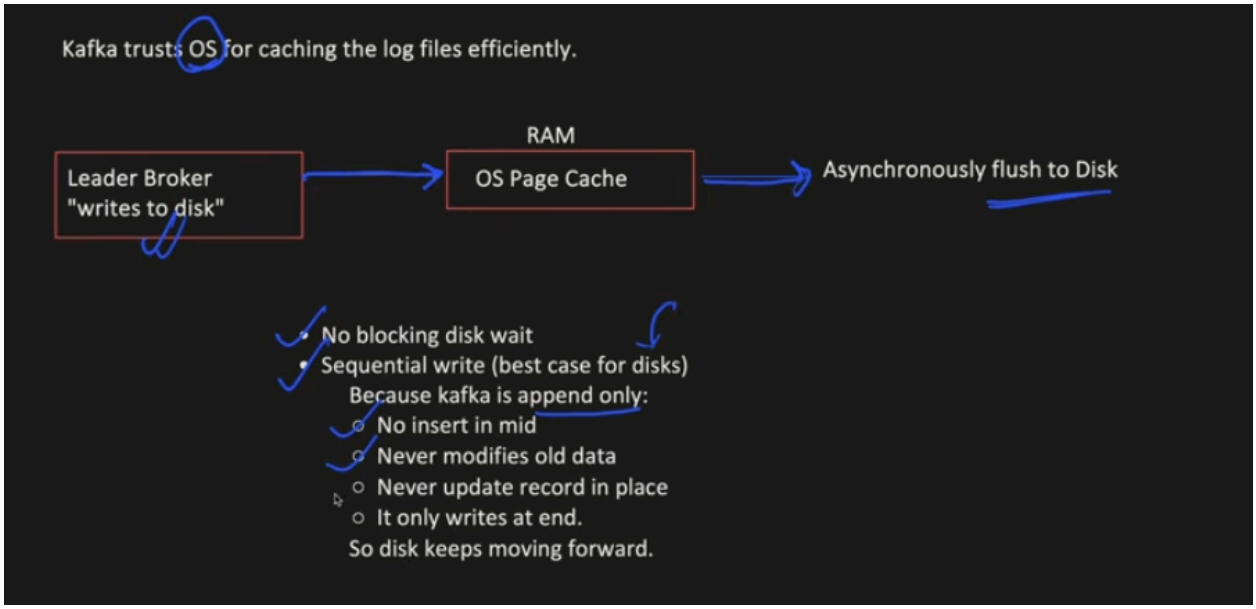
Disk → Page Cache → Application → Network

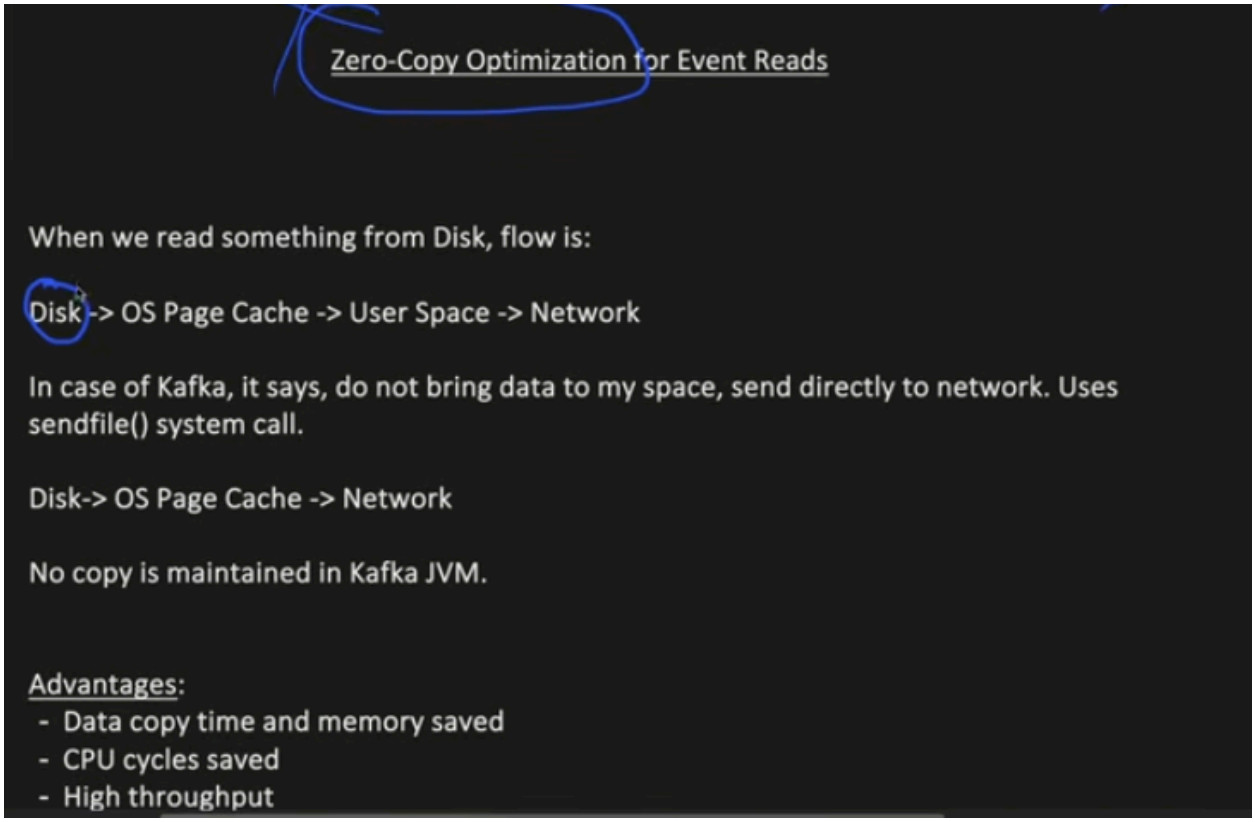
Kafka uses:

Disk → Page Cache → Network

Benefits:

- Lower CPU usage
- Faster throughput
- Less memory usage





---

28. Failure Scenarios

Controller Failure

If active controller crashes:

1. Standby controllers detect heartbeat loss
2. Election starts
3. New controller elected via quorum

Cluster continues working.

Leader Partition Failure

If leader broker fails:

1. Controller detects failure
2. New leader elected from ISR
3. Metadata updated
4. Clients reconnect

Result:

No data loss

Follower Failure

Follower removed from ISR.

System continues normally.

When follower recovers:

- It catches up
- Rejoins ISR

Broker Failure

Controller:

- Elects new leaders
- Updates metadata

System continues.

Topic Failure

Topic never fails.

---

29. Minimum In Sync Replicas

min.insync.replicas = 2

If ISR count drops below this:

Partition becomes unavailable

This prevents **data loss**.